

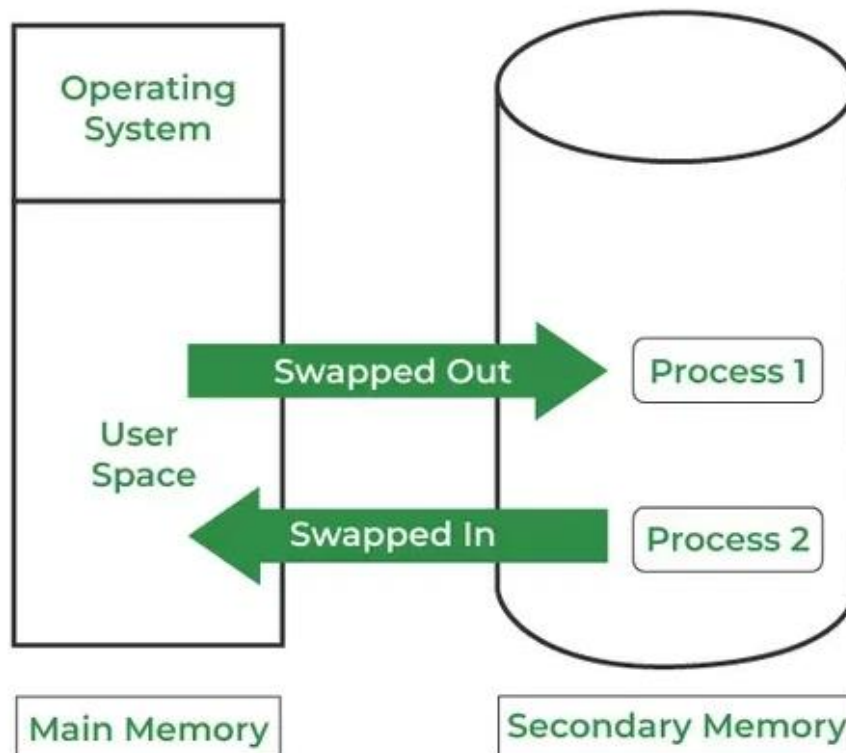
UNIT IV: MEMORY MANAGEMENT

Swapping

Swapping is a memory management technique in which a process is temporarily moved from main memory (RAM) to secondary storage (disk) and vice versa. This allows the operating system to manage limited RAM effectively and run multiple processes concurrently in a multiprogramming environment.

- Reduces memory fragmentation by moving processes in and out of RAM.
- Allows large processes to run even if they exceed available RAM.
- Frees RAM temporarily by suspending inactive or waiting processes.
- Supports virtual memory to give processes more memory than physically available.

Example: RAM has 4 GB with Process A and Process B running, each using 2 GB. When Process C, which also requires 2 GB, needs to run, there is no free memory. The operating system swaps Process A out to the disk and loads Process C into RAM. After Process C finishes, Process A is brought back into memory to continue execution.



Process Swapping in Operating Systems

How it Works:

1. A process is moved into RAM from disk when it is ready to execute.
2. After the process has run for a while (or if higher-priority processes need memory), it may be temporarily swapped out to disk.
3. The CPU scheduler decides which process to swap in and out, often based on priority or scheduling policies.
4. When the swapped-out process is needed again, it is swapped back into memory to continue execution.

Note: *Swapping moves whole processes between RAM and disk, which is slow and inefficient. Modern OS like Windows, Linux, and macOS use paging instead of swapping. Paging loads only the needed parts of a process, making memory management faster and efficient.*

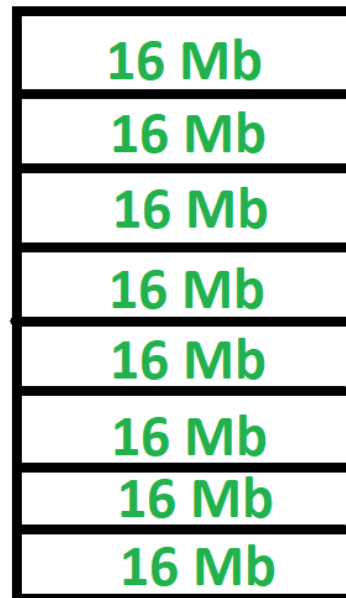
Advantages	Disadvantages
Reduces waiting time by using swap space as an extension of RAM, keeping the CPU busy.	Risk of data loss if the system crashes during swapping.
Frees main memory by moving inactive processes to secondary storage.	Increased page faults due to frequent swapping reduce performance.
Allows multiple processes to run using a single main memory with a swap partition.	Disk access is slower than RAM, causing execution delays.
Enables large programs to run on systems with limited physical memory.	Excessive swapping (thrashing) causes severe performance degradation.
Prevents memory overload by ensuring active processes have enough RAM.	CPU may spend more time swapping than executing processes.

Difference between Fixed Partitioning and Variable Partitioning

In multi-programming, the most important methods of memory management include Fixed and Variable Partitioning. Under Fixed Partitioning memory is divided into partitions which are fixed whereas under Variable Partitioning memory partition is varied based on process size. Each has its advantages and disadvantages as to flexibility, efficiency, and the ease of using the system and is therefore appropriate for a given system.

What is Fixed Partitioning?

Fixed Partitioning is a contiguous memory management technique in which the main memory is divided into fixed sized partitions which can be of equal or unequal size. Whenever we have to allocate a process memory then a free partition that is big enough to hold the process is found. Then the memory is allocated to the process. If there is no free space available then the process waits in the queue to be allocated memory. It is one of the oldest memory management techniques which is easy to implement.



Advantages of Fixed Partitioning

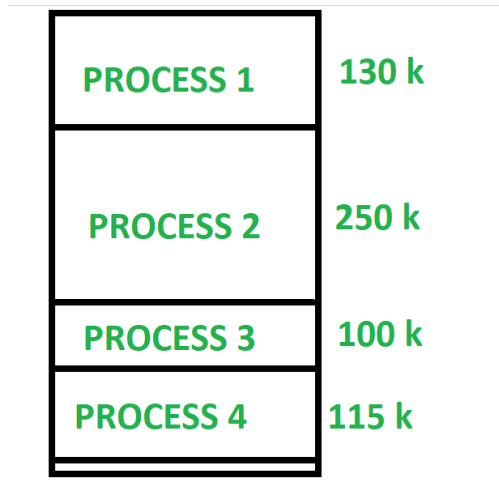
- **Simplicity:** Since partitions are already defined, it is easy to assign memory to a process especially when it is designed that way.
- **Ease of Implementation:** This makes partitioning method very easy to implement especially since it only requires partitioning of a few segments.

Disadvantages of Fixed Partitioning

- **Internal Fragmentation:** If a process is not able to fill a partition it uses then there is space left in that partition which is not utilized.
- **Inefficient Memory Utilization:** The size of partitions is fixed and therefore large processes might occupy more space while small processes occupy inadequate large space.
- **Limited Process Size:** Quantities greater than the size of the largest partition cannot be performed; this decreases flexibility.

What is Variable Partitioning?

Variable Partitioning is a contiguous memory management technique in which the main memory is not divided into partitions and the process is allocated a chunk of free memory that is big enough for it to fit. The space which is left is considered as the free space which can be further used by other processes. It also provides the concept of compaction. In compaction the spaces that are free and the spaces which not allocated to the process are combined and single large memory space is made.



Advantages of Variable Partitioning

- **Efficient Memory Utilization:** Memory is allocated dynamically; this means that it can be used sparingly thus avoiding the use of much space on the available memory.
- **No Process Size Limitation:** One advantage of this is an absence of sharp partitions because memory is allocated as per the processes requirements.
- **Higher Degree of Multi-programming:** It also stated that process flexibility for memory can be suitable for more process.

Disadvantages of Variable Partitioning

- **External Fragmentation:** Any time memory space is granted and then recalled for someone else to free, space of maximum un-utilized portions of memory are created termed as external fragmentation leading to poor memory utilization.
- **Complexity:** Fixed partitioning and its related concepts are far simpler to handle and control as compared to variable partitions and external fragmentation.
- **Need for Compaction:** To avoid this fragmentation, compaction is needed, in which the free areas are grouped together to form bigger spaces in the memory, but this takes a lot of time and consumes many resources.

Difference between Fixed Partitioning and Variable Partitioning

Fixed Partitioning	Variable Partitioning
Memory is divided into fixed-sized partitions.	Memory is allocated dynamically in varying sizes.
Only one process can be placed in each partition.	A process is allocated a chunk of free memory as needed.
Inefficient memory utilization due to internal fragmentation.	More efficient memory utilization with less internal fragmentation.
Internal and external fragmentation occur.	Only external fragmentation occurs.
Limited degree of multi-programming.	Higher degree of multi-programming due to flexible allocation.
Easier to implement.	More complex to implement.
Restricts process size based on partition size.	No size limitation on processes.

Partition Allocation Methods

Memory management techniques are methods used by an operating system to allocate, control, and release main memory during program execution. They ensure that multiple processes can use memory efficiently without interfering with each other. Proper memory management improves system performance and prevents memory wastage.

- **Single contiguous allocation:** Simplest allocation method used by MS-DOS. All memory (except some reserved for OS) is available to a process.
- **Partitioned allocation:** Memory is divided into different blocks or partitions. Each process is allocated according to the requirement.
- **Paged memory management:** Memory is divided into fixed-sized units called page frames, used in a virtual memory environment.
- **Segmented memory management:** Memory is divided into different segments (a segment is a logical grouping of the process' data or code). In this management, allocated memory doesn't have to be contiguous.

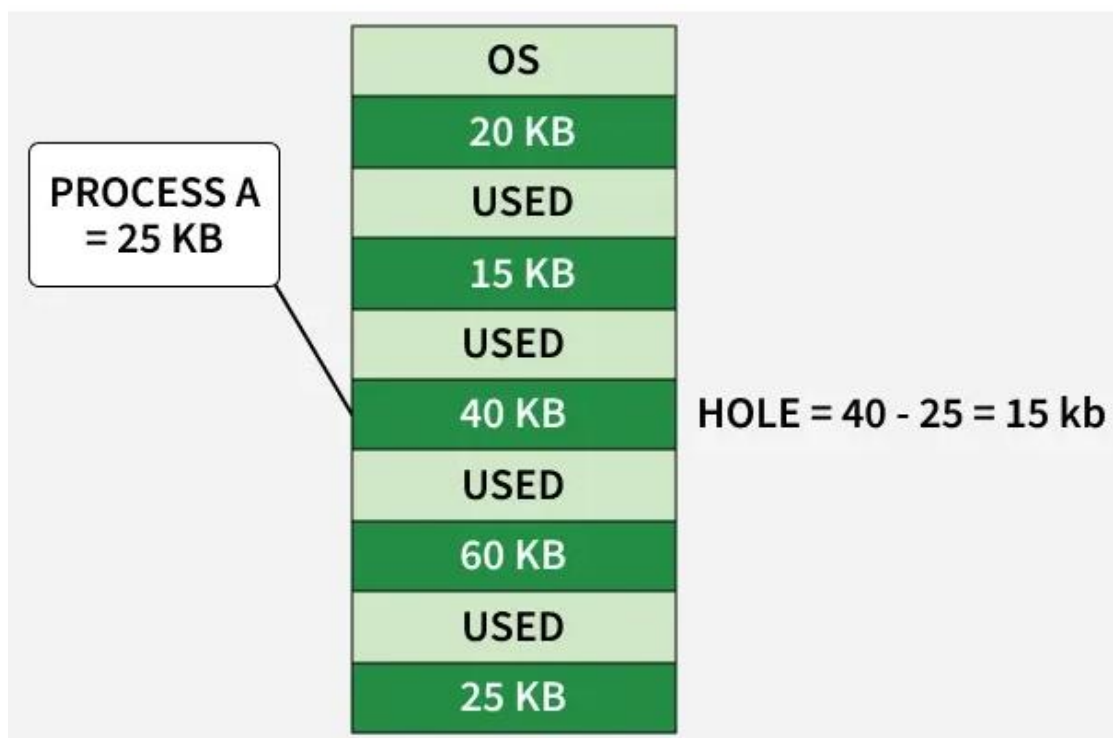
Most of the operating systems (for example Windows and Linux) use Segmentation with Paging. A process is divided into segments and individual segments have pages.

*In **Partition Allocation**, when there is more than one partition freely available to accommodate a process's request, a partition must be selected. To choose a particular partition, a partition allocation method is needed. A partition allocation method is considered better if it avoids internal fragmentation.*

There are different Placement Algorithm:

1. First Fit
2. Best Fit
3. Worst Fit
4. Next Fit

1. First Fit: In the first fit, the partition is allocated which is the first sufficient block from the top of Main Memory. It scans memory from the beginning and chooses the first available block that is large enough. Thus it allocates the first hole that is large enough.



2. Best Fit Allocate the process to the partition which is the first smallest sufficient partition among the free available partition. It searches the entire list of holes to find the smallest hole whose size is greater than or equal to the size of the process.



3. Worst Fit Allocate the process to the partition which is the largest sufficient among the freely available partitions available in the main memory. It is opposite to the best-fit algorithm. It searches the entire list of holes to find the largest hole and allocate it to process.



4. Next Fit: Next fit is similar to the first fit but it will search for the first sufficient partition from the last allocation point.

Is Best-Fit really best?

Although best fit minimizes the wastage space, it consumes a lot of processor time for searching the block which is close to the required size. Also, Best-fit may perform poorer than other algorithms in some cases. For example, see the below exercise.

Comparison of Partition Allocation Methods:

Sl.No.	Partition Allocation Method	Advantages	Disadvantages
1.	Fixed Partition	Simple, easy to use, no complex algorithms needed	Memory waste, inefficient use of memory resources
2.	Dynamic Partition	Flexible, more efficient, partitions allocated as required	Requires complex algorithms for memory allocation
3.	Best-fit Allocation	Minimizes memory waste, allocates smallest suitable partition	More computational overhead to find smallest split
4.	Worst-fit Allocation	Ensures larger processes have sufficient memory	May result in substantial memory waste
5.	First-fit Allocation	Quick, efficient, less computational work	Risk of memory fragmentation

Exercise:

Consider the memory requests from processes in the following order:

300K, 25K, 125K, 50K.

The available memory blocks are:

150K, 150K, 350K.

Which of the following partition allocation schemes can satisfy the above requests?

- A) Best fit but not first fit
- B) First fit but not best fit
- C) Both first fit & best fit
- D) Neither first fit nor best fit

Solution:

Best Fit:

1. 300K → allocated from 350K block, leaving 50K.
2. 25K → allocated from the remaining 50K block, leaving 25K.
3. 125K → allocated from 150K block, leaving 25K.
4. 50K → cannot be allocated (remaining free spaces are 25K + 25K, which is insufficient).

Best fit cannot satisfy all requests.

First Fit:

1. 300K → allocated from 350K block, leaving 50K.
2. 25K → allocated from 150K block, leaving 125K.
3. 125K → allocated from the remaining 125K block.
4. 50K → allocated from the remaining 50K block.

First fit can satisfy all requests.

Answer:

B) First fit but not best fit

Demand Paging

Demand paging is a technique used in virtual memory systems where pages enter main memory only when requested or needed by the CPU. OS loads only the necessary pages of a program into memory at runtime, instead of loading the entire program into memory at the start. Here,

- A page fault occurred when the program needed to access a page that is not currently in memory.
- The operating system then loads the required pages from the disk into memory and updates the page tables accordingly.
- This process is transparent to the running program and it continues to run as if the page had always been in memory.

Pure Demand Paging

Pure demand paging is a specific implementation of demand paging. In on-demand paging only, no pages are initially loaded into memory when the program starts and all pages are initially marked as being on disk.

- Operating systems that use pure demand paging as a memory management strategy do so without preloading any pages into physical memory prior to the commencement of a task.
- Demand paging loads a process's whole address space into memory one step at a time, bringing just the parts of the process that are actively being used into memory from disc as needed.
- It is useful for executing huge programs that might not fit totally in memory or for computers with limited physical memory.
- If the program accesses a lot of pages that are not in memory right now, it could also result in a rise in page faults and possible performance overhead.
- Operating systems frequently use caching techniques and improve page replacement algorithms to lessen the negative effects of page faults on system performance as a whole.

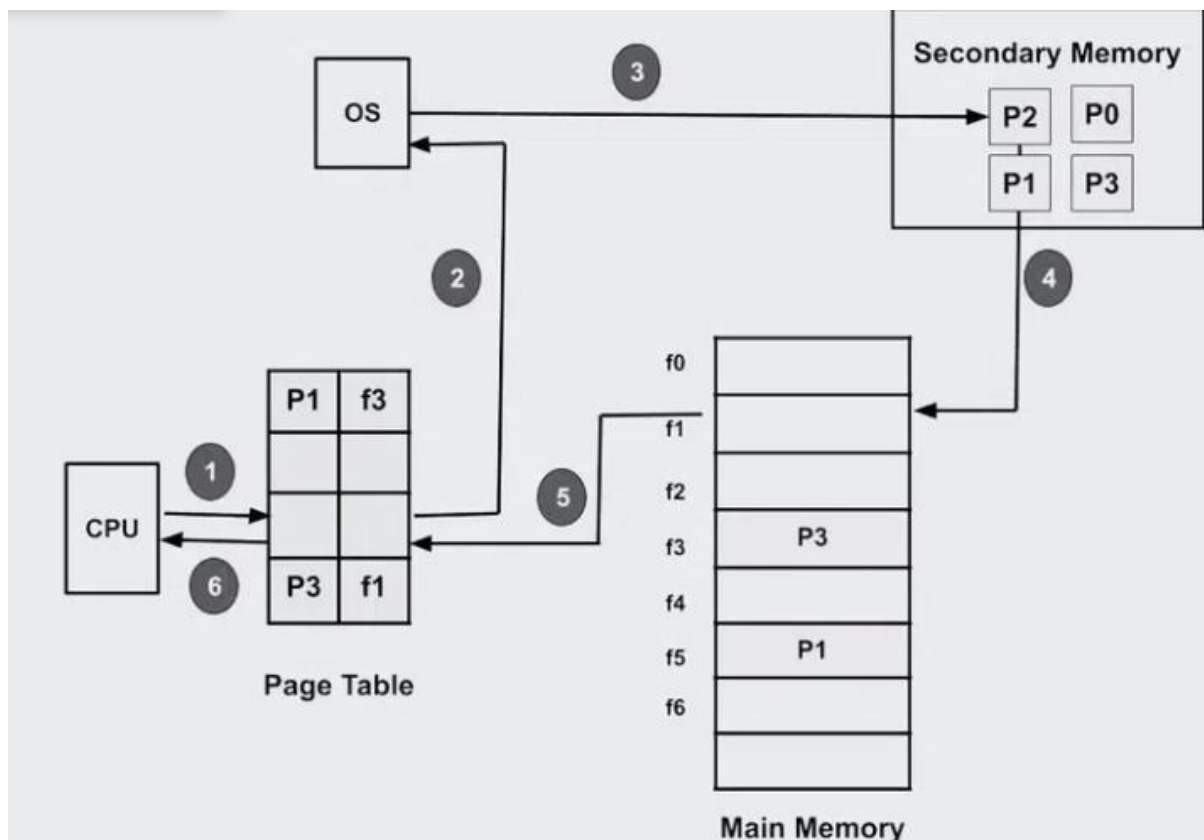
Working Process of Demand Paging

Let us understand this with the help of an example. Suppose we want to run a process P which have four pages P0, P1, P2 and P3. Currently, in the page table, we have pages P1 and P3.

The operating system's demand paging mechanism follows a few steps in its operation:

- **Program Execution:** Upon launching a program, the operating system allocates a certain amount of memory to the program and establishes a process for it.
- **Creating Page Tables:** To keep track of which program pages are currently in memory and which are on disk, the operating system makes page tables for each process.
- **Handling Page Fault:** When a program tries to access a page that isn't in memory at the moment, a page fault happens. In order to determine whether the necessary page is on disk, the operating system pauses the application and consults the page tables.
- **Page Fetch:** The operating system loads the necessary page into memory by retrieving it from the disk if it is there.
- The page's new location in memory is then reflected in the page table.
- **Resuming The Program:** The operating system picks up where it left off when the necessary pages are loaded into memory.

- **Page Replacement:** If there is not enough free memory to hold all the pages a program needs, the operating system may need to replace one or more pages currently in memory with pages currently in memory. on the disk. The page replacement algorithm used by the operating system determines which pages are selected for replacement.
- **Page Cleanup:** When a process terminates, the operating system frees the memory allocated to the process and cleans up the corresponding entries in the page tables.



How Demand Paging in OS Affects System Performance?

Demand paging can improve system performance by reducing the memory needed for programs and allowing multiple programs to run simultaneously.

- If not implemented properly, it can cause performance issues.
- When a program needs a part that isn't in the main memory, the operating system must fetch it from the hard disk, which takes time and pauses the program.
- This can cause delays and if the system runs out of memory, it will need to frequently swap pages in and out, increasing delays and reducing performance.

Page Replacement Algorithms

In an operating system that uses paging, a page replacement algorithm is needed when a page fault occurs and no free page frame is available. In this case, one of the existing pages in memory must be replaced with the new page.

The virtual memory manager performs this by:

1. Selecting a victim page using a page replacement algorithm.
2. Marking its page table entry as “not present.”
3. If the page was modified (dirty), writing it back to disk before replacement.

The efficiency of a page replacement algorithm directly affects the page fault rate, which in turn impacts system performance.

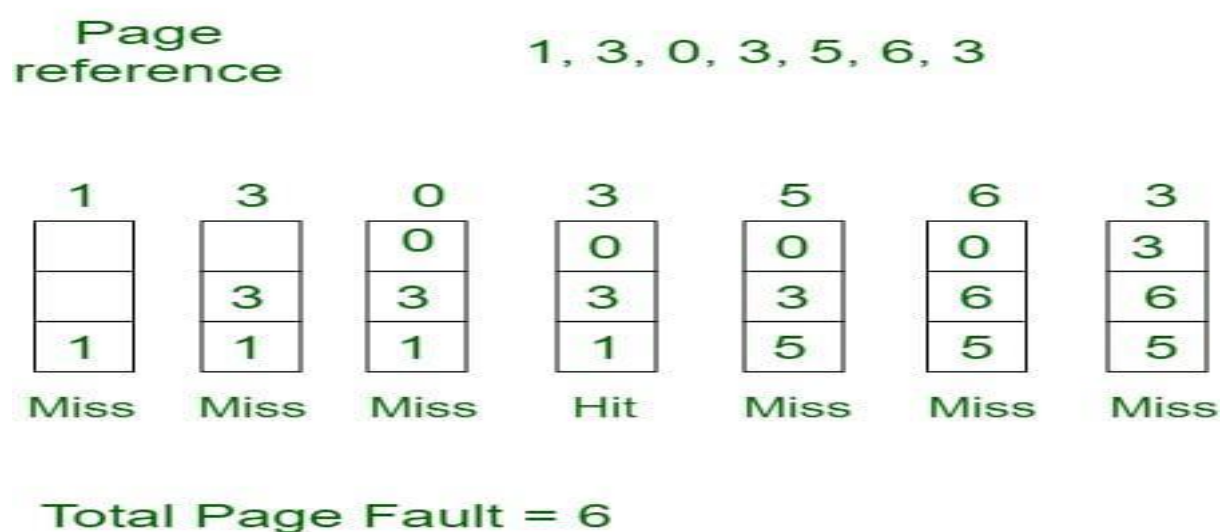
Common Page Replacement Techniques

- First In First Out (FIFO)
- Optimal Page replacement
- Least Recently Used (LRU)

1. First in First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.



FIFO - Page Replacement

- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots ---> **3 Page Faults.**
- When 3 comes, it is already in memory so ---> **0 Page Faults.**
- Then 5 comes, it is not available in memory, so it replaces the oldest page slot i.e 1. ---> **1 Page Fault.**
- 6 comes, it is also not available in memory, so it replaces the oldest page slot i.e 3 ---> **1 Page Fault.**
- Finally, when 3 come it is not available, so it replaces 0 1-page **fault.**

2. Optimal Page Replacement

In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

Example: Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3														No. of Page frame - 4													
7	0	1	2	0	3	0	4	2	3	0	3	2	3															
			2	2	2	2	2	2	2	2	2	2	2															
		1	1	1	1	1	4	4	4	4	4	4	4															
	0	0	0	0	0	0	0	0	0	0	0	0	0															
7	7	7	7	7	3	3	3	3	3	3	3	3	3															
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit															

Total Page Fault = 6

Optimal Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future---> 1 Page fault.
- 0 is already there so ---> 0 Page fault. 4 will takes place of 1 ---> 1 Page Fault.

Now for the further page reference string ---> 0 Page fault because they are already available in the memory. Optimal page replacement is perfect, but not possible in practice as the

operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analyzed against it.

3. Least Recently Used

In this algorithm, page will be replaced which is least recently used.

Example Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using LRU Page Replacement Algorithm.

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3														No. of Page frame - 4	
7	0	1	2	0	3	0	4	2	3	0	3	2	3			
			2	2	2	2	2	2	2	2	2	2	2			
		1	1	1	1	1	4	4	4	4	4	4	4			
	0	0	0	0	0	0	0	0	0	0	0	0	0			
7	7	7	7	7	3	3	3	3	3	3	3	3	3			
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit			
Total Page Fault = 6																

Here LRU has same number of page fault as optimal but it may differ according to question.

Least Recently Used - Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults.
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is least recently used ---> 1 Page fault.
- 0 is already in memory so ---> 0 Page fault.
- 4 will takes place of 1 ---> 1 Page Fault.
- Now for the further page reference string ---> 0 Page fault because they are already available in the memory.